Trie → String based operation.

prefix tree

Hash table → search O(1)

String: len L → O(L)

Suffix tree.

ab cde.

phonebook^

very good at string retrieval.

algo
also
algea
also

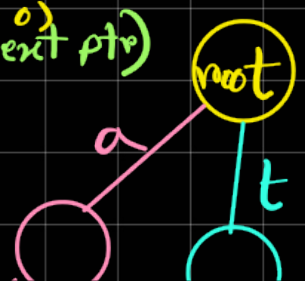
toy
tom
to

'a' - 'a' = 0
'b' - 'a' = 1
⋮
z - 'a' = 25

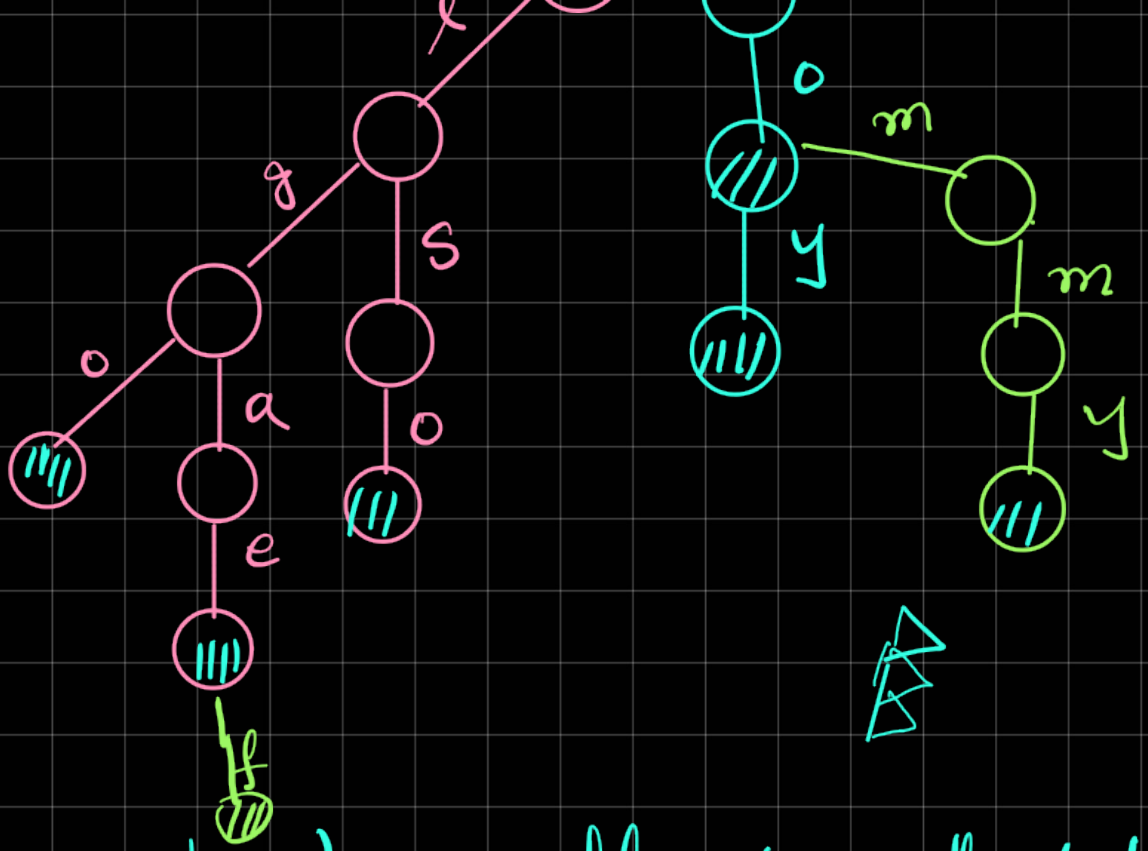
we want lower case letters

so, every node will have node next[26]

at 1st no a^, so allocate memory for a^ (next of 0) (next ptr)




||| → endmark




doesn't physically store the data
just the pointers to the data.

word len

To search Algorithm → search O(L)
next pointer to ro
0 so not present.

endmark ☆ flag.

how many names has ike ✗
trie not efficient for substring.

Startswith function:

prefix_count.

Algo:

```
Struct Node{
 boolean endmark;
 Node * next[26];

Node(){
   endmark = fake
   for(i=0 — 26)
     Next[i] = 'NULL'
   }
}

void insert(Str, len){
   Node * cur = root
for(i=0 ——— len)
  idx = Str[i] - 'a';
if(cur → next[idx] == NULL)
|    cur → next[idx] = new Node()
```

```
cur->countEP++;
cur = cur->next[idx];
```

→ for prefix count

```
cur->endmark = True;
```

$O(NL)$ → Time complexity.

$O(N \times L \times Alphabet_Size)$

↓

26

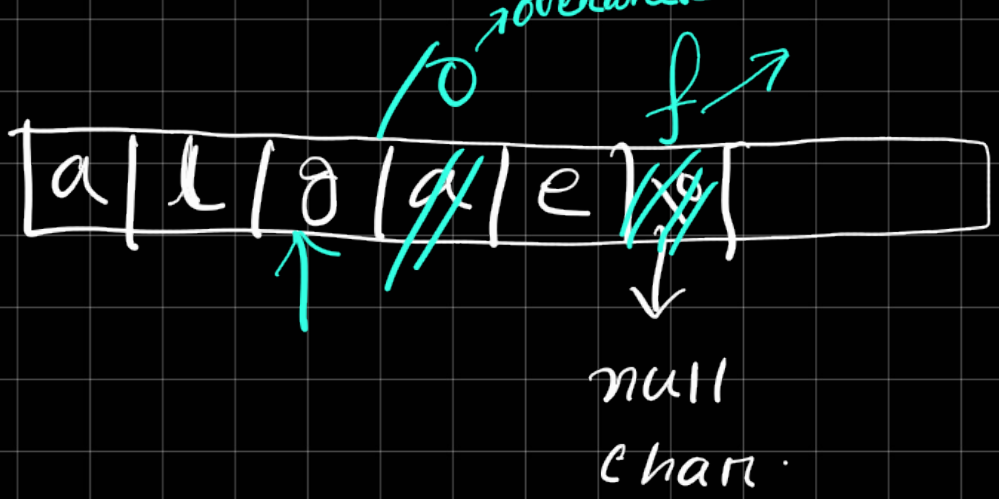
```
bool search (str, len){
    Node * cur = root;
    for (i=0 — len)
    {
        idx = str[i] - 'a';
        if (cur->next [idx]==NULL)
            return false;
        cur = cur->next[idx];
    }
```

return cur->endmark;

O(L) → complexity.

## print func:

each node has separate status of the func → recursive implementation.




a

l

o

a

e



flexible (start from any)

```
void display ( Node * curr, Str[],
               level ) → int : remembers
                           corresponding
                           idx (prev)

if (curr → endmark == True)
    Str [level] = '\0';
    cout << Str;
for (i=0 ⟶ 26)
    if ( curr → next [i] != NULL)
        ch = 'a' + i;
        Str [level] = ch;
        display (curr → next [i],
                 Str , level + 1);
```

---



root, —, 0

a — a — 1

l, al, 2

g, alg, 3

a, alga, 4

e, algae, 5 ✓

f, algaef, 6

0, algo, 4

S also, 4

0, also, 5

0 1 2 3 4 5 6

S 0

algae ✓

algaef

Store binary numbers in a trie.

Next wednesday → Hashing, trie, + Disjoint set.

13th weekend → 4th quiz.

Lab Final.

* Phonebook autocomplete
* given a set of numbers.
  - Find max XOR between apair.